

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#04)

Use of Venus Simulator for Register Level Execution of RISC-V Arithmetic and Memory Instructions

Student Name: Manahil Shahzad

Roll No: 24I-6563

Section: CE-A

Date performed: 8TH September, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Task 1:

Description:

In this task, we basically took two registers s0 and s1 and stored two values in it. We then applied the basic arithmetic operations on it to see if we get the correct result, we stimulated it on the venus stimulator and ultimately got the correct result.

Code:

```
.text

li s0,5

li s1,4

add s2,s0,s1

sub s3,s0,s1

mul s4,s0,s1

mul s5,s0,s0
```

Output:

PC	Machine Code	Basic Code	Original Code	tp (x4)
0x0	0x00500413	addi x8 x0 5	li s0,5	0
0x4	0x00400493	addi x9 x0 4	li s1,4	0
0x8	0x00940933	add x18 x8 x9	add s2,s0,s1	0
0xc	0x409409B3	sub x19 x8 x9	sub s3,s0,s1	0
0x10	0x02940A33	mul x20 x8 x9	mul s4,s0,s1	0
0x14	0x02040AB3	mul x21 x8 x8	mul s5,s0,s0	0

PC	Machine Code	Basic Code	Original Code	tp (x4)
0x0	0x00500413	addi x8 x0 5	li s0,5	0
0x4	0x00400493	addi x9 x0 4	li s1,4	0
0x8	0x00940933	add x18 x8 x9	add s2,s0,s1	0
0xc	0x409409B3	sub x19 x8 x9	sub s3,s0,s1	0
0x10	0x02940A33	mul x20 x8 x9	mul s4,s0,s1	0
0x14	0x02040AB3	mul x21 x8 x8	mul s5,s0,s0	0

PC	Machine Code	Basic Code	Original Code	a7 (x17)
0x0	0x00500413	addi x8 x0 5	li s0,5	0
0x4	0x00400493	addi x9 x0 4	li s1,4	0
0x8	0x00940933	add x18 x8 x9	add s2,s0,s1	0
0xc	0x409409B3	sub x19 x8 x9	sub s3,s0,s1	0
0x10	0x02940A33	mul x20 x8 x9	mul s4,s0,s1	0
0x14	0x02840AB3	mul x21 x8 x8	mul s5,s0,s0	0

PC	Machine Code	Basic Code	Original Code	a7 (x17)
0x0	0x00500413	addi x8 x0 5	li s0,5	0
0x4	0x00400493	addi x9 x0 4	li s1,4	0
0x8	0x00940933	add x18 x8 x9	add s2,s0,s1	0
0xc	0x409409B3	sub x19 x8 x9	sub s3,s0,s1	1
0x10	0x02940A33	mul x20 x8 x9	mul s4,s0,s1	0
0x14	0x02840AB3	mul x21 x8 x8	mul s5,s0,s0	0

PC	Machine Code	Basic Code	Original Code	a7 (x17)
0x0	0x00500413	addi x8 x0 5	li s0,5	0
0x4	0x00400493	addi x9 x0 4	li s1,4	0
0x8	0x00940933	add x18 x8 x9	add s2,s0,s1	0
0xc	0x409409B3	sub x19 x8 x9	sub s3,s0,s1	1
0x10	0x02940A33	mul x20 x8 x9	mul s4,s0,s1	20
0x14	0x02840AB3	mul x21 x8 x8	mul s5,s0,s0	20

PC	Machine Code	Basic Code	Original Code	a7 (x17)
0x0	0x00500413	addi x8 x0 5	li s0,5	0
0x4	0x00400493	addi x9 x0 4	li s1,4	0
0x8	0x00940933	add x18 x8 x9	add s2,s0,s1	1
0xc	0x409409B3	sub x19 x8 x9	sub s3,s0,s1	20
0x10	0x02940A33	mul x20 x8 x9	mul s4,s0,s1	20
0x14	0x02840AB3	mul x21 x8 x8	mul s5,s0,s0	20

Task 2:

Description:

In this lab, we took 5 values that were stored in the register and then we performed the following operations as indicated in the lab task. We first used the sub command along with addition and then multiplication and stimulated it on the Venus stimulator and our result was therefore verified.

Code:

```
.text

li s0,5

li s1,4

li s2,3

li s3,2

li s4,1

sub s5,s0,s1

add s6,s2,s3

mul s7,s6,s4

add s8,s5,s7
```

Output:

PC	Machine Code	Basic Code	Original Code	s2 (x7)
0x0	0x00500413	addi x8 x0 5	li s0,5	
0x4	0x00400493	addi x9 x0 4	li s1,4	
0x8	0x00300913	addi x18 x0 3	li s2,3	
0xc	0x00200993	addi x19 x0 2	li s3,2	
0x10	0x00100A13	addi x20 x0 1	li s4,1	
0x14	0x40940AB3	sub x21 x8 x9	sub s5,s0,s1	
0x18	0x01990B33	add x22 x18 x19	add s6,s2,s3	
0x1c	0x034B0BB3	mul x23 x22 x20	mul s7,s6,s4	
0x20	0x017AB333	add x24 x21 x23	add s8,s5,s7	

PC	Machine Code	Basic Code	Original Code	
0x0	0x00500413	addi x8 x0 5	li a0,5	a2 (x18)
0x4	0x00400493	addi x9 x0 4	li a1,4	a3 (x19)
0x8	0x00300913	addi x18 x0 3	li a2,3	a4 (x20)
0xc	0x00200993	addi x19 x0 2	li a3,2	a5 (x21)
0x10	0x00100A13	addi x20 x0 1	li a4,1	a6 (x22)
0x14	0x40940AB3	sub x21 x8 x9	sub a5,a0,a1	a7 (x23)
0x18	0x01390B33	add x22 x18 x19	add a6,a2,a3	a8 (x24)
0x1c	0x034B0WB3	mul x23 x22 x20	mul a7,a6,a4	
0x20	0x017A8C33	add x24 x21 x23	add a8,a5,a7	
Copy! Download! Clear!				

Task 3:

Description:
In this task, two numbers were taken and then they were stored in two registers s0 and s1. Three operations and, or and xor were performed on it and we then verified our results on the Venus stimulator and they were proved right.
Code:
<pre>.text li s0,0xffff li s1,0xf0f and s2,s0,s1 or s3,s0,s1 xor s4,s0,s1</pre>
Output:

PC	Machine Code	Basic Code	Original Code	
0x0	0x00001437	lui x8 1	li s0,0xffff	t0 (x5) 4
0x4	0xFFFF40413	addi x8 x8 -1	li s0,0xffff	t1 (x6) 4
0x8	0x000014B7	lui x9 1	li s1,0xf0f	t2 (x7) 8
0xc	0xF0F48493	addi x9 x9 -241	li s1,0xf0f	s0 (x8) 4096
0x10	0x00947933	and x18 x8 x9	and s2,s0,s1	s1 (x9) 3456
0x14	0x009469B3	or x19 x8 x9	or s3,s0,s1	a0 (x10) 8
0x18	0x00944A33	xor x20 x8 x9	xor s4,s0,s1	

PC	Machine Code	Basic Code	Original Code	
0x0	0x00001437	lui x8 1	li s0,0xffff	(x16)
0x4	0xFFFF40413	addi x8 x8 -1	li s0,0xffff	a7 (x17) 8
0x8	0x000014B7	lui x9 1	li s1,0xf0f	s2 (x18) 5056
0xc	0xF0F48493	addi x9 x9 -241	li s1,0xf0f	s3 (x19) 4096
0x10	0x00947933	and x18 x8 x9	and s2,s0,s1	s4 (x20) 3456
0x14	0x009469B3	or x19 x8 x9	or s3,s0,s1	
0x18	0x00944A33	xor x20 x8 x9	xor s4,s0,s1	

Task 4:

Description:
In this task, an array of integers and its size were passed to the program through registers a0 and a1. The program uses a loop to access each element of the array, add all the elements together, and store the final sum in register t0.
Code:
<pre>.text .globl arraysum arraysum: # a0 = int a[] # a1 = int size # t0 = ret # t1 = i</pre>

```

li    t0, 0      # ret = 0

li    t1, 0      # i = 0

loop: # For loop

    bge t1, a1, done    # if i >= size, break

    slli t2, t1, 2    # Multiply i by 4 (1 << 2 = 4)

    add  t2, a0, t2    # Update memory address

    lw   t2, 0(t2)    # Dereference address to get integer

    add  t0, t0, t2    # Add integer value to ret

    addi t1, t1, 1    # Increment the iterator

    j loop # Jump back to start of loop (1 backwards)

done:

    mv   a0, t0    # Move t0 (ret) into a0

    ret  # Return via return address register

```

Output:

PC	Machine Code	Basic Code	Original Code	Regs
0x0	0x00000133	addi a1, a0, 0	li a0, 0 # ret = 0	ra (x1)
0x4	0x00000133	addi a1, a0, 0	li a1, 0 # i = 0	sp (x2)
0x8	0x00000133	bge a1, a2, done	bge a1, a2, done # if i >= size, break	gp (x3)
0xc	0x00000133	slli a7, a1, 2	slli a2, a1, 2 # Multiply i by 4 (1 << 2 = 4)	fp (x4)
0x10	0x007f0133	add a7, a2, a7	add a2, a0, a2 # Update memory address	tp (x5)
0x14	0x00000133	lw a7, 0(a7)	lw a2, 0(a2) # Dereference address to get integer	t1 (x6)
0x18	0x007f0133	add a0, a0, a7	add a0, a0, a2 # Add integer value to ret	t2 (x7)
0x1c	0x00000133	addi a1, a1, 1	addi a1, a1, 1 # Increment the iterator	tp (x8)
0x20	0x00000133	j loop	j loop # Jump back to start of loop (1 backwards)	tp (x9)
0x24	0x00000133	addi a0, a0, 0	mv a0, a0 # Move a0 (ret) into a0	tp (x10)
0x28	0x00000133	ret	ret # Return via return address register	tp (x11)

PC	Machine Code	Basic Code	Original Code	Regs
0x0	0x00000133	addi a1, a0, 0	li a0, 0 # ret = 0	ra (x1)
0x4	0x00000133	addi a1, a0, 0	li a1, 0 # i = 0	sp (x2)
0x8	0x00000133	bge a1, a2, done	bge a1, a2, done # if i >= size, break	gp (x3)
0xc	0x00000133	slli a7, a1, 2	slli a2, a1, 2 # Multiply i by 4 (1 << 2 = 4)	fp (x4)
0x10	0x007f0133	add a7, a2, a7	add a2, a0, a2 # Update memory address	tp (x5)
0x14	0x00000133	lw a7, 0(a7)	lw a2, 0(a2) # Dereference address to get integer	t1 (x6)
0x18	0x007f0133	add a0, a0, a7	add a0, a0, a2 # Add integer value to ret	t2 (x7)
0x1c	0x00000133	addi a1, a1, 1	addi a1, a1, 1 # Increment the iterator	tp (x8)
0x20	0x00000133	j loop	j loop # Jump back to start of loop (1 backwards)	tp (x9)
0x24	0x00000133	addi a0, a0, 0	mv a0, a0 # Move a0 (ret) into a0	tp (x10)
0x28	0x00000133	ret	ret # Return via return address register	tp (x11)

PC	Machine Code	Basic Code	Original Code	name	Comment
0x0	0x00000201	addi v0 v0 0	li t0, 0 # ret = 0	r0 (x0)	0x00000000
0x4	0x00000113	addi v0 v0 0	li t1, 0 # i = 0	sp (x2)	0x00000000
0x8	0x00031003	lgs v0 v11 20	lgs t1, a1, done # if i > size, break	gp (x3)	0x00000000
0xc	0x00331103	slli v7 v0 2	slli t2, t1, 2 # Multiply i by 4 (1 << 2 = 4)	tp (x4)	0x00000000
0x10	0x00730301	add v7 v10 v7	add t2, a0, t2 # Update memory address	fp (x5)	0x00000000
0x14	0x00031003	lw v7 0(v7)	lw t2, 0(t2) # Dereference address to get integer	t0 (x6)	0x00000000
0x18	0x00730301	add v0 v0 v7	add t0, t0, t2 # Add integer value to ret	t1 (x7)	0x00000000
0x1c	0x00130313	addi v0 v0 1	addi t1, t1, 1 # Increment the iterator	v0 (x8)	0x00000000
0x20	0x00000007	jals v0 -24	j loop # Jump back to start of loop (1 backwards)	v1 (x9)	0x00000000
0x24	0x00031013	addi v10 v0 0	mv a0, t0 # Move t0 (ret) into a0	v0 (x10)	0x00000000
0x28	0x00000007	jals v0 v1 0	ret # Return via return address register	v1 (x11)	0x00000000

PC	Machine Code	Basic Code	Original Code	name	Comment
0x0	0x00000201	addi v0 v0 0	li t0, 0 # ret = 0	r0 (x0)	0x00000000
0x4	0x00000113	addi v0 v0 0	li t1, 0 # i = 0	sp (x2)	0x00000000
0x8	0x00031003	lgs v0 v11 20	lgs t1, a1, done # if i > size, break	gp (x3)	0x00000000
0xc	0x00331103	slli v7 v0 2	slli t2, t1, 2 # Multiply i by 4 (1 << 2 = 4)	tp (x4)	0x00000000
0x10	0x00730301	add v7 v10 v7	add t2, a0, t2 # Update memory address	fp (x5)	0x00000000
0x14	0x00031003	lw v7 0(v7)	lw t2, 0(t2) # Dereference address to get integer	t0 (x6)	0x00000000
0x18	0x00730301	add v0 v0 v7	add t0, t0, t2 # Add integer value to ret	t1 (x7)	0x00000000
0x1c	0x00130313	addi v0 v0 1	addi t1, t1, 1 # Increment the iterator	v0 (x8)	0x00000000
0x20	0x00000007	jals v0 -24	j loop # Jump back to start of loop (1 backwards)	v1 (x9)	0x00000000
0x24	0x00031013	addi v10 v0 0	mv a0, t0 # Move t0 (ret) into a0	v0 (x10)	0x00000000
0x28	0x00000007	jals v0 v1 0	ret # Return via return address register	v1 (x11)	0x00000000

PC	Machine Code	Basic Code	Original Code	name	Comment
0x0	0x00000201	addi v0 v0 0	li t0, 0 # ret = 0	r0 (x0)	0
0x4	0x00000113	addi v0 v0 0	li t1, 0 # i = 0	sp (x2)	0x00000000
0x8	0x00031003	lgs v0 v11 20	lgs t1, a1, done # if i > size, break	gp (x3)	0x00000000
0xc	0x00331103	slli v7 v0 2	slli t2, t1, 2 # Multiply i by 4 (1 << 2 = 4)	tp (x4)	0
0x10	0x00730301	add v7 v10 v7	add t2, a0, t2 # Update memory address	fp (x5)	0
0x14	0x00031003	lw v7 0(v7)	lw t2, 0(t2) # Dereference address to get integer	t0 (x6)	0
0x18	0x00730301	add v0 v0 v7	add t0, t0, t2 # Add integer value to ret	t1 (x7)	0
0x1c	0x00130313	addi v0 v0 1	addi t1, t1, 1 # Increment the iterator	v0 (x8)	0
0x20	0x00000007	jals v0 -24	j loop # Jump back to start of loop (1 backwards)	v1 (x9)	0
0x24	0x00031013	addi v10 v0 0	mv a0, t0 # Move t0 (ret) into a0	v0 (x10)	0
0x28	0x00000007	jals v0 v1 0	ret # Return via return address register	v1 (x11)	0x00000000

PC	Machine Code	Basic Code	Original Code	name	Comment
0x0	0x00000201	addi v0 v0 0	li t0, 0 # ret = 0	r0 (x0)	0
0x4	0x00000113	addi v0 v0 0	li t1, 0 # i = 0	sp (x2)	0x00000000
0x8	0x00031003	lgs v0 v11 20	lgs t1, a1, done # if i > size, break	gp (x3)	0x00000000
0xc	0x00331103	slli v7 v0 2	slli t2, t1, 2 # Multiply i by 4 (1 << 2 = 4)	tp (x4)	0
0x10	0x00730301	add v7 v10 v7	add t2, a0, t2 # Update memory address	fp (x5)	0
0x14	0x00031003	lw v7 0(v7)	lw t2, 0(t2) # Dereference address to get integer	t0 (x6)	0
0x18	0x00730301	add v0 v0 v7	add t0, t0, t2 # Add integer value to ret	t1 (x7)	0x00000000
0x1c	0x00130313	addi v0 v0 1	addi t1, t1, 1 # Increment the iterator	v0 (x8)	0
0x20	0x00000007	jals v0 -24	j loop # Jump back to start of loop (1 backwards)	v1 (x9)	0
0x24	0x00031013	addi v10 v0 0	mv a0, t0 # Move t0 (ret) into a0	v0 (x10)	0
0x28	0x00000007	jals v0 v1 0	ret # Return via return address register	v1 (x11)	0x00000000

PC	Machine Code	Basic Code	Original Code	PC	Machine Code	Basic Code	Original Code
0x0	0x00000213	addi s0 s0 0	li s0, 0 # ret = 0	0x1	0x00000213	addi s0 s0 0	li s0, 0 # ret = 0
0x4	0x00000313	addi s0 s0 0	li s1, 0 # i = 0	0x5	0x00000313	addi s0 s0 0	li s1, 0 # i = 0
0x8	0x00033263	lqr s0 s11 28	lqr s1, s1, done # if i is size, break	0x9	0x00033263	lqr s0 s11 28	lqr s1, s1, done # if i is size, break
0xc	0x00231393	slll s7 s0 2	slll s2, s1, 2 # Multiply i by 4 (2 << 2 = 4)	0xd	0x00231393	slll s7 s0 2	slll s2, s1, 2 # Multiply i by 4 (2 << 2 = 4)
0x0	0x00750303	add s7 s10 s7	add s2, s0, s1 # Update memory address	0x1	0x00750303	add s7 s10 s7	add s2, s0, s1 # Update memory address
0x4	0x00033303	lw s7 0(s7)	lw s2, 0(s2) # Dereference address to get integer	0x5	0x00033303	lw s7 0(s7)	lw s2, 0(s2) # Dereference address to get integer
0x8	0x00750303	add s5 s5 s7	add s0, s0, s2 # Add integer value to ret	0x9	0x00750303	add s5 s5 s7	add s0, s0, s2 # Add integer value to ret
0xc	0x00130313	addi s0 s0 1	addi s1, s1, 1 # Increment the iterator	0xd	0x00130313	addi s0 s0 1	addi s1, s1, 1 # Increment the iterator
0x0	0x00ff000f	jlt s0 -20	j loop # Jump back to start of loop (1 backwards)	0x1	0x00ff000f	jlt s0 -20	j loop # Jump back to start of loop (1 backwards)
0x4	0x00033313	addi s20 s0 0	sw s0, s0 # Store s0 (ret) into s0	0x5	0x00033313	addi s20 s0 0	sw s0, s0 # Store s0 (ret) into s0
0x8	0x00000007	jlr s0 s1 0	ret # Return via return address register	0x9	0x00000007	jlr s0 s1 0	ret # Return via return address register

PC	Machine Code	Basic Code	Original Code	PC	Machine Code	Basic Code	Original Code
0x0	0x00000213	addi s0 s0 0	li s0, 0 # ret = 0	0x1	0x00000213	addi s0 s0 0	li s0, 0 # ret = 0
0x4	0x00000313	addi s0 s0 0	li s1, 0 # i = 0	0x5	0x00000313	addi s0 s0 0	li s1, 0 # i = 0
0x8	0x00033263	lqr s0 s11 28	lqr s1, s1, done # if i is size, break	0x9	0x00033263	lqr s0 s11 28	lqr s1, s1, done # if i is size, break
0xc	0x00231393	slll s7 s0 2	slll s2, s1, 2 # Multiply i by 4 (2 << 2 = 4)	0xd	0x00231393	slll s7 s0 2	slll s2, s1, 2 # Multiply i by 4 (2 << 2 = 4)
0x0	0x00750303	add s7 s10 s7	add s2, s0, s1 # Update memory address	0x1	0x00750303	add s7 s10 s7	add s2, s0, s1 # Update memory address
0x4	0x00033303	lw s7 0(s7)	lw s2, 0(s2) # Dereference address to get integer	0x5	0x00033303	lw s7 0(s7)	lw s2, 0(s2) # Dereference address to get integer
0x8	0x00750303	add s5 s5 s7	add s0, s0, s2 # Add integer value to ret	0x9	0x00750303	add s5 s5 s7	add s0, s0, s2 # Add integer value to ret
0xc	0x00130313	addi s0 s0 1	addi s1, s1, 1 # Increment the iterator	0xd	0x00130313	addi s0 s0 1	addi s1, s1, 1 # Increment the iterator
0x0	0x00ff000f	jlt s0 -20	j loop # Jump back to start of loop (1 backwards)	0x1	0x00ff000f	jlt s0 -20	j loop # Jump back to start of loop (1 backwards)
0x4	0x00033313	addi s20 s0 0	sw s0, s0 # Store s0 (ret) into s0	0x5	0x00033313	addi s20 s0 0	sw s0, s0 # Store s0 (ret) into s0
0x8	0x00000007	jlr s0 s1 0	ret # Return via return address register	0x9	0x00000007	jlr s0 s1 0	ret # Return via return address register

PC	Machine Code	Basic Code	Original Code	PC	Machine Code	Basic Code	Original Code
0x0	0x00000213	addi s0 s0 0	li s0, 0 # ret = 0	0x1	0x00000213	addi s0 s0 0	li s0, 0 # ret = 0
0x4	0x00000313	addi s0 s0 0	li s1, 0 # i = 0	0x5	0x00000313	addi s0 s0 0	li s1, 0 # i = 0
0x8	0x00033263	lqr s0 s11 28	lqr s1, s1, done # if i is size, break	0x9	0x00033263	lqr s0 s11 28	lqr s1, s1, done # if i is size, break
0xc	0x00231393	slll s7 s0 2	slll s2, s1, 2 # Multiply i by 4 (2 << 2 = 4)	0xd	0x00231393	slll s7 s0 2	slll s2, s1, 2 # Multiply i by 4 (2 << 2 = 4)
0x0	0x00750303	add s7 s10 s7	add s2, s0, s1 # Update memory address	0x1	0x00750303	add s7 s10 s7	add s2, s0, s1 # Update memory address
0x4	0x00033303	lw s7 0(s7)	lw s2, 0(s2) # Dereference address to get integer	0x5	0x00033303	lw s7 0(s7)	lw s2, 0(s2) # Dereference address to get integer
0x8	0x00750303	add s5 s5 s7	add s0, s0, s2 # Add integer value to ret	0x9	0x00750303	add s5 s5 s7	add s0, s0, s2 # Add integer value to ret
0xc	0x00130313	addi s0 s0 1	addi s1, s1, 1 # Increment the iterator	0xd	0x00130313	addi s0 s0 1	addi s1, s1, 1 # Increment the iterator
0x0	0x00ff000f	jlt s0 -20	j loop # Jump back to start of loop (1 backwards)	0x1	0x00ff000f	jlt s0 -20	j loop # Jump back to start of loop (1 backwards)
0x4	0x00033313	addi s20 s0 0	sw s0, s0 # Store s0 (ret) into s0	0x5	0x00033313	addi s20 s0 0	sw s0, s0 # Store s0 (ret) into s0
0x8	0x00000007	jlr s0 s1 0	ret # Return via return address register	0x9	0x00000007	jlr s0 s1 0	ret # Return via return address register

Task 5:

Description:

In this task, the table of 2 was initialized in the data section using .data. The values from 2 to 20 were stored in the table. These values were then loaded into different registers from s0 to s9 using the lw instruction. The program was run and verified in the Venus simulator, and the results were correct.

Code:

```
.data
```

table: .word 2,4,6,8,10,12,14,16,18,20

.text

.globl main

main:

la t0, table

lw s0, 0(t0)

lw s1, 4(t0)

lw s2, 8(t0)

lw s3, 12(t0)

lw s4, 16(t0)

lw s5, 20(t0)

lw s6, 24(t0)

lw s7, 28(t0)

lw s8, 32(t0)

lw s9, 36(t0)

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x1000327	lwpc r5, 0(0)	la r0, table
0x4	0x0001233	addi r5, r5, 0	la r0, table
0x8	0x0001003	la r0, 0(r0)	la r0, 0(r0)
0xc	0x0001003	la r0, 4(r0)	la r0, 4(r0)
0x0	0x0001003	la r0, 8(r0)	la r0, 8(r0)
0x4	0x0001003	la r0, 12(r0)	la r0, 12(r0)
0x8	0x0001003	la r0, 16(r0)	la r0, 16(r0)
0xc	0x0001003	la r0, 20(r0)	la r0, 20(r0)
0x0	0x0001003	la r0, 24(r0)	la r0, 24(r0)
0x4	0x0001003	la r0, 28(r0)	la r0, 28(r0)
0x8	0x0001003	la r0, 32(r0)	la r0, 32(r0)
0xc	0x0001003	la r0, 36(r0)	la r0, 36(r0)

0x10	0x01234567	1w x11 20(x5)	1w x6, 20(x6)		
0x18	0x01234567	1w x11 20(x5)	1w x7, 20(x6)	x8 {x11}	16
0x20	0x00000000	1w x10 12(x5)	1w x8, 12(x6)	x8 {x22}	11
0x28	0x00000000	1w x10 12(x5)	1w x8, 12(x6)	x7 {x23}	6
0x30	0x00000000	1w x10 16(x5)	1w x8, 16(x6)	x8 {x26}	6

0x24	0x05C2A8B3	3e x23 20(x5)	3e x7, 28(x0)	0x 1(x0)	10
0x28	0x0206A7B3	3e x24 32(x3)	3e x8, 32(x0)	x8 (x22)	20
0x2c	0x0242A7B3	3e x25 36(x5)	3e x9, 36(x0)	x7 (x23)	20
				x8 (x24)	20
0x24	0x05C2A8B3	3e x23 20(x5)	3e x7, 28(x0)	x8 (x22)	20
0x28	0x0206A7B3	3e x24 32(x3)	3e x8, 32(x0)	x8 (x22)	20
0x2c	0x0242A7B3	3e x25 36(x5)	3e x9, 36(x0)	x7 (x23)	20
				x8 (x24)	20